The easiest formula to add momentum into our update rule for points can look like this:

$$P_{t+1} \leftarrow \beta * P_t - \alpha * (1-\beta) * \nabla g(P_t)$$

Where P_t is a point in the problem space, specifically the point reached by the algorithm at time t. The higher is beta, the smoother (and slower) the update will be:

$$P_2 \leftarrow \beta * P_1 - \alpha * (1-\beta) * \nabla g(P_1) = \beta^2 * P_0 - \alpha * \beta * (1-\beta) * \nabla g(P_0) - \alpha * (1-\beta) * \nabla g(\beta * P_1 - \alpha * (1-\beta) * \nabla g(P_1))$$

So, after 2 steps, if $\beta=0.99$ then 98% of the value of P_2 is given by P_0; conversely, if $\beta=0.1$, P_0 directly influences P_2 only for 1%.

16.4 Applications of Gradient Descent

As mentioned, gradient descent is an optimization technique that, given a cost function, helps finding a solution (a point in the problem space) to which it corresponds an optimal (or nearly optimal) cost.

As such, it is only a piece of the process of solving a problem, and at the same time it can be applied to several different problems and techniques.

The overall algorithm depends first, as we have seen, on the cost function used, but also on the goal of the optimization.